

Web-First Apple Music Synchronized Listening Platform

Product thesis, technical architecture, development roadmap, constraints, validation gates and expansion strategy.

North star Enable legitimate Apple Music subscribers on different supported devices to join one private room and experience coordinated playback. Apple delivers every participant's audio; this platform coordinates the experience.		
Current scope	Primary client	Later expansion
Private prototype for personal/friends use and portfolio demonstration	Web first using MusicKit on the Web	Native Android, Android TV/Google TV, then other clients only after web validation

Baseline version 1.0 | September 2026

1. Executive Decision

Development begins with the web version. The first objective is not a complete consumer product and not the Android TV client. The first objective is to experimentally prove that MusicKit on the Web supplies the authorization, full-track playback, seeking and playback-state primitives required for a synchronization engine.

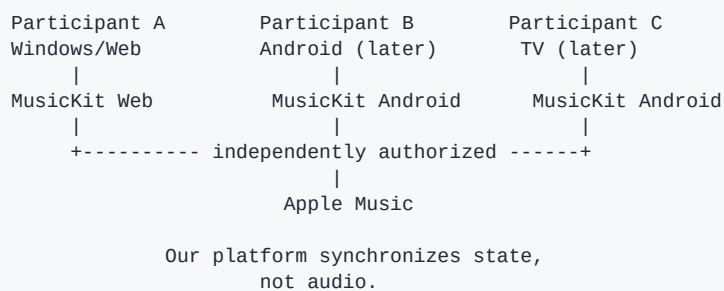
If the web proof succeeds, the same room protocol and synchronization model become the foundation for native Android and Android TV clients. Native clients are extensions of the platform, not replacements for the web product.

Development rule

Do not expand the feature set until the current phase passes its acceptance gate with real Apple Music subscriber sessions.

2. Problem Statement

Apple Music subscribers can use different device ecosystems. The project targets private shared listening where each participant already has legitimate Apple Music access but wants a common room, queue and playback timeline across supported devices. The system does not create a new audio distribution path.



The platform is

- A room and coordination service.
- A distributed playback synchronization engine.
- A social/party layer around independently authorized Apple Music sessions.
- A web-first product with future native clients.

The platform is not

- A streaming service, audio proxy or Apple Music replacement backend.
- A way to give non-subscribers free access.
- A downloader, recorder, transcoder, DRM bypass or stream extractor.
- A literal port or modification of Apple's consumer applications.
- Initially a public venue, commercial broadcast or multi-provider service.

3. Core Product Modes

Mode	Purpose	Playback model
Remote synchronized room	Friends in different places listen together	Every participant independently streams Apple Music; the room coordinates timing.
In-person party, later	Phones/web clients collaborate around one room output	A TV or selected device can be the primary output while guests request/vote/control.
Personal TV client, later	Independent Apple Music experience on Android TV	Native TV client using official Apple interfaces where technically supported.

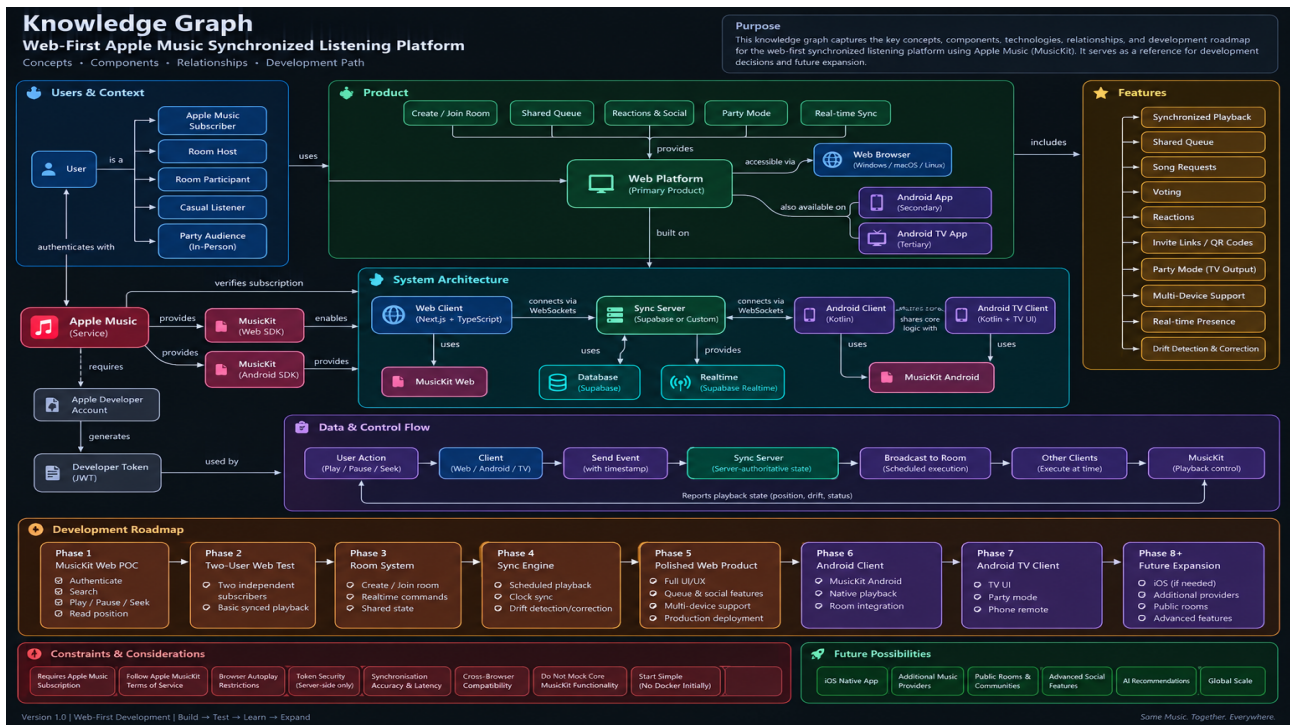
4. Platform Boundary

Our platform owns	Apple / MusicKit owns
Rooms, invites, presence and host authority	Apple Music account authorization and entitlement
Shared queue and room state	Catalog/media service
Clock synchronization and scheduled commands	DRM and audio delivery
Drift measurement and correction decisions	Actual MusicKit-rendered playback
Reconnect, sequence ordering and telemetry	Provider-side availability and service behavior

The backend must never receive, cache, transform or redistribute Apple Music audio.

5. Knowledge Graph

The following map summarizes the product, clients, Apple integration, synchronization flow, roadmap and constraints.



The web platform remains the primary entry point. Android and Android TV reuse the room protocol and synchronization concepts after the web implementation is proven.

6. Initial Technology Stack

Layer	Decision	Rationale
Web	Next.js + TypeScript	Fast iteration, server routes for token work, deployable web product.
UI	React + Tailwind CSS	Minimal POC first; polished room UI later.
Apple playback	MusicKit on the Web	Official browser playback path.
Apple data	Apple Music API	Catalog search and later user/library functionality.
Realtime	Supabase Realtime initially	Low-operations prototype path.
Database	Supabase PostgreSQL	Rooms/queue persistence when needed.
Validation	Zod	Typed realtime/API contracts.
Testing	Vitest + real browser/device tests	Pure sync logic can be automated; MusicKit behavior must be validated on real sessions.
Deployment	Vercel or equivalent	Simple web deployment.
Containers	No Docker initially	It does not reduce the primary MusicKit/synchronization risks.

7. Security and Apple Integration

- Use official MusicKit and Apple Music API interfaces only.

- Generate/sign Apple developer tokens in trusted server-side code. Never ship the Media Services private key to the browser.
- Every listener independently authorizes Apple Music and must have legitimate playback entitlement.
- Never log sensitive authorization tokens, private keys, cookies or Authorization headers.
- Use HTTPS/WSS, unpredictable private room identifiers and server-side validation of host authority/events.
- Review current Apple documentation and applicable developer terms again before public or commercial distribution.

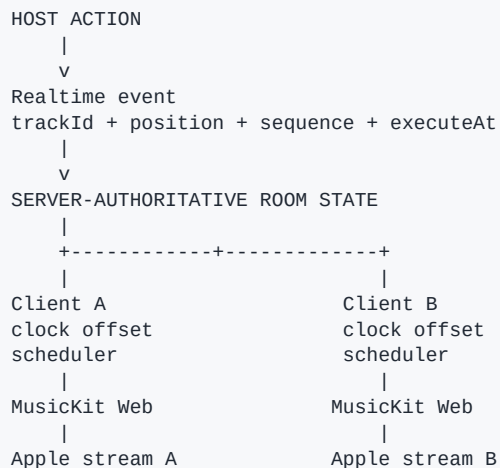
Apple's current documentation describes MusicKit playback on websites and Apple Music API developer tokens signed with ES256. Current documentation should be treated as the implementation authority because API details and terms can change.

8. Player Abstraction

```
interface SyncPlayer {  
  load(trackId)  
  play()  
  pause()  
  seek(positionMs)  
  getPosition()  
  getDuration()  
  getState()  
}  
  
Web implementation:  
SyncPlayer -> MusicKit Web  
  
Later:  
SyncPlayer -> MusicKit Android
```

The synchronization engine must not contain platform-specific branches throughout its logic. Platform differences belong in adapters.

9. Synchronization Architecture



Canonical state

```
CanonicalRoomState {
    roomId
    sequence
    trackId
    playbackState
    positionAtEventMs
    eventServerTimeMs
    hostParticipantId
}

If PLAYING:
    expectedPositionMs =
        positionAtEventMs +
        (estimatedServerNowMs - eventServerTimeMs)
```

Why scheduled execution

Immediate PLAY messages arrive at different times. Commands should normally target a future server timestamp so clients can receive, prepare and execute against a common time reference. Late clients should derive the current expected position instead of executing stale timing.

Clock synchronization

Each client estimates its offset from server time using repeated request/response timestamp samples. RTT and clock offset are measured continuously enough to support scheduled actions and recovery. High-latency samples should not dominate the estimate.

Drift

During playback, each client compares the expected room position with the position reported by its local MusicKit player. Small drift is tolerated; persistent or large drift can trigger corrective seeking. Thresholds are experimental and must be tuned from measured results, not assumed.

10. Realtime Protocol

Event	Purpose
ROOM_JOIN / ROOM_LEAVE	Presence and membership
TRACK_LOAD / TRACK_READY	Prepare the same catalog resource
PLAY / PAUSE / SEEK / SKIP	Shared playback state changes
ROOM_STATE	Canonical recovery snapshot
PLAYER_STATUS	Playing, paused, buffering, disconnected or degraded
SYNC_REPORT	Position, drift, RTT/offset and resync telemetry
HOST_CHANGED	Predictable authority transfer

State-changing events use monotonic sequence numbers. Clients must reject stale events.

11. Required Debug Telemetry

Room	ABC829
Sequence	184
Room state	PLAYING
RTT	52 ms
Clock offset	+18 ms
Expected position	83.420 s
Actual position	83.382 s
Current drift	-38 ms
Median drift	61 ms
P95 drift	174 ms
Resync count	2
Player state	PLAYING

The debug view is a required part of the prototype. The project must demonstrate measured synchronization rather than only visually or audibly appearing synchronized.

12. Development Phases

Phase	Build	Exit gate
0 - MusicKit Web POC	Authorize, catalog track, full playback, play/pause/seek, position/state	One legitimate subscriber works reliably in Chrome/Edge.
1 - Two-user validation	Two independent Apple Music subscribers play the same catalog track	Independent entitlement and playback verified.
2 - Room system	Private room, host/guest, realtime events, presence	Host actions reach both clients reliably.
3 - Canonical state	Server-authoritative playback state and sequence ordering	Reconnect can reconstruct current state.
4 - Clock + scheduling	RTT/offset estimator and future-timestamp commands	Clients execute against common server time.
5 - Drift engine	Measure expected vs actual position; conservative correction	Drift distribution and resync behavior are measurable.
6 - Web product	Invite links, search, queue, participant states, responsive UI	Repeatable friends-only web experience.
7 - Android	Native MusicKit Android adapter and client	Windows/web and Android share one room.
8 - Android TV	TV feasibility, TV UI, phone remote/party mode	TV becomes a supported client/output if MusicKit works reliably.
9 - Future	iOS if useful, other providers only after policy/API research	Separate decision.

13. Phase 0 Exact Deliverable

The first implementation should be intentionally unattractive and diagnostic.

MUSICKIT WEB POC

[Connect Apple Music]
Authorization:

Search: [_____]
Result:

[Load] [Play] [Pause]
[Seek -10] [Seek +10]

Position: 00:42.381
Player state: PLAYING
Last error: none

Phase 0 go/no-go

Do not build rooms or social UI until full authorized playback, seeking and sufficiently useful position/state reporting are verified on real subscriber sessions.

14. Web Product UX After Technical Validation

Surface	Minimum experience
Landing	Create room, join room, concise subscriber requirement.
Join	Invite link/code, nickname, Connect Apple Music, readiness.
Room	Artwork, track/progress, host controls, participants and queue.
Search	Apple Music catalog search and add/select.
Presence	Ready, playing, buffering, disconnected, resyncing.
Debug	Hideable technical synchronization metrics.

Browser autoplay restrictions mean users should explicitly interact with the page before synchronized playback is armed. Native apps later improve background/media behavior but are not required to prove the web concept.

15. Minimal Data Model

Entity	Fields
rooms	id, code, host_participant_id, status, created_at
participants	id, room_id, nickname, client_platform, joined_at, last_seen_at
room_state	room_id, track_id, playback_state, position_at_event_ms, event_server_time_ms, sequence
queue	id, room_id, track_id, added_by, queue_position, created_at

Do not persist every playback sample. High-frequency telemetry should remain ephemeral or be sampled for test analysis.

16. Major Challenges

Challenge	Severity	Approach
MusicKit Web behavior across browsers	High	Validate before product work; maintain capability/error reporting.
Authorization and entitlement	High	Every listener authorizes independently; explicit failure states.
Network latency/jitter	High	Clock estimation and scheduled execution.
Playback drift	Critical	Continuous measurement and conservative correction.
Buffering/startup variance	High	Prepare/load before scheduled start where supported.
Background tab/autoplay	High	Explicit user gesture, test throttling/recovery.
Reconnect	High	Canonical room state and state reconstruction.
Out-of-order commands	High	Monotonic sequence numbers and validation.
Bluetooth/output latency	External	Separate physical output latency from player-position drift.
Regional catalog differences	Medium	Handle unavailable tracks/storefronts gracefully.
Apple API/terms changes	High	Official APIs only; revalidate before wider distribution.
Premature expansion	High	No Android/TV/other providers until web gates pass.

17. Test Plan

- Chrome Windows to Chrome Windows using two independent subscribers.
- Chrome to Edge on Windows.
- Same network and different networks.
- Normal latency and intentionally poor/high-jitter network conditions.
- Foreground versus background tab.
- Temporary disconnect and reconnect.
- Host refresh/disconnect and authority recovery.
- Rapid play/pause/seek input and stale-event rejection.
- At least one approximately 30-minute listening session with drift distribution and resync count recorded.
- Later: Windows/web to Android, then Android TV on physical hardware.

18. Success Criteria

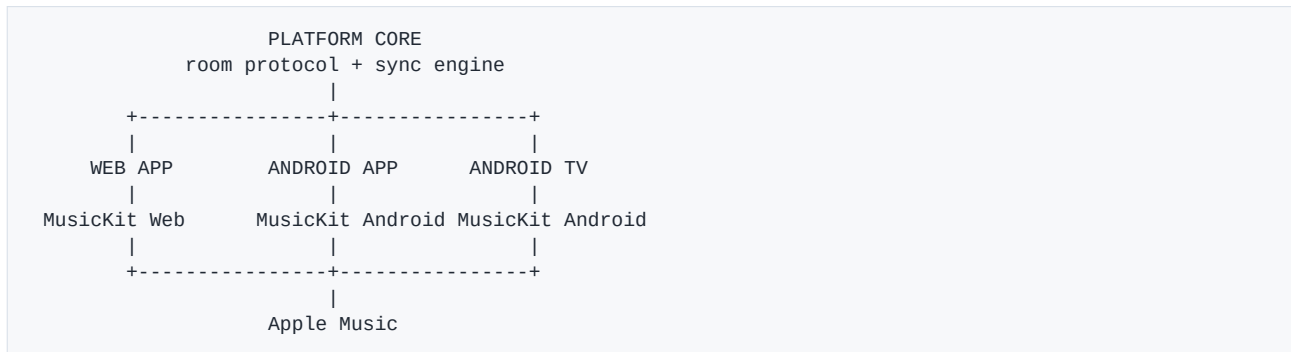
Level	Definition
Feasibility	One subscriber can authorize and expose the required MusicKit Web playback primitives.
Independent playback	Two subscribers independently render the same Apple Music track.
Realtime	Shared controls reliably propagate.
Synchronized execution	Clients schedule actions against a common time reference.
Measured quality	Expected vs actual position and drift are continuously observable.
Reliability	Reconnect and stale-event scenarios recover predictably.
Showcase	A repeatable cross-client demo presents real measured statistics.

Do not claim a fixed synchronization accuracy before real measurements. Exploratory targets such as sub-150 ms median absolute player drift and sub-400 ms P95 may guide testing, but measured MusicKit/browser behavior is authoritative.

19. Docker Decision

Docker is deliberately excluded from the initial development environment. The first risks are MusicKit authentication, playback control, browser behavior and synchronization. Use local Node.js/pnpm, a lockfile and environment variables. Introduce containers only if a later dedicated sync server, Redis, PostgreSQL or self-hosted infrastructure creates a concrete reproducibility/operations need.

20. Expansion Strategy



Android

After the web sync engine is proven, implement a native Android SyncPlayer adapter and reuse the room protocol. This validates the strongest cross-device use case: Windows/web and Android in one room.

Android TV

After Android works, run a dedicated MusicKit-on-Android-TV feasibility test before building a full TV client. If successful, the TV application can provide both an independent Apple Music TV experience and party mode, with phone/web clients acting as controllers or synchronized participants.

Other providers

Additional music services are explicitly deferred. They introduce provider-specific playback APIs, policies, catalog mapping and licensing constraints. The Apple Music-only implementation should remain clean until its synchronization model is proven.

21. Team Working Model

Web / distributed systems	Native clients later
Next.js, MusicKit Web, token service, rooms, realtime protocol, canonical state, clock sync, scheduling, drift telemetry and web UI.	Kotlin, MusicKit Android, Android playback adapter, native media behavior, Android UI, Android TV feasibility and TV UI.

Both sides must agree on the room protocol and SyncPlayer behavior. The native client should not require redesigning the synchronization protocol.

22. Development Operating Rules

- Build one phase at a time and preserve measured results from each phase.
- Do not mock the critical MusicKit feasibility gate. Unit mocks are fine, but acceptance requires real Apple Music playback.
- Record discovered limitations rather than hiding them behind UI.
- Keep secrets out of client code and Git history.
- Keep architecture simple until complexity solves a demonstrated problem.
- Prefer official provider capabilities over reverse-engineered shortcuts.
- Treat the synchronization engine as the core engineering asset.
- Keep the web join experience installation-free.
- Do not choose additional providers or production scaling technology during the initial proof.

23. Phase Handoff Template

After each phase, record the following so the next development step is based on evidence:

Field	Record
Phase	Number and name
Implemented	Features and code paths completed
Environment	Browser/device/OS/network tested
Apple APIs used	Exact MusicKit/API version and relevant methods
Measured results	RTT, offset, drift, failure/recovery observations
Known issues	Reproducible bugs and platform limitations
Architecture changes	Any deviation from this baseline and why
Security/config	New environment variables or service dependencies
Tests	Automated and manual cases passed/failed
Next recommendation	Proceed, revise, or stop the assumption

Working agreement

Use this final ideation as the baseline. After every development phase, update the evidence and decisions before starting the next phase.

24. Final Project Statement

Build a web-first, private synchronized listening platform that coordinates independently authenticated Apple Music playback sessions across supported devices while leaving subscription entitlement, DRM and audio delivery entirely to Apple. Prove the synchronization system on the web first; then extend the same platform to native Android and Android TV.

The project's technical value is not the playback UI. It is the combination of server-authoritative room state, clock-offset estimation, scheduled execution, event ordering, recovery, drift measurement and cross-client playback adapters.

25. Official Reference Starting Points

MusicKit: <https://developer.apple.com/musickit/>

Apple Music API: <https://developer.apple.com/documentation/applemusicapi>

Developer Tokens: <https://developer.apple.com/documentation/applemusicapi/generating-developer-tokens>

Media identifier/private key setup:

<https://developer.apple.com/help/account/capabilities/create-a-media-identifier-and-private-key>

Before implementing or distributing, verify the current Apple documentation and terms. This document is a product/engineering baseline, not legal advice.